

✓ PCA From Scratch trên UCI HAR — Giai đoạn 1 đến 4

Mục tiêu: xây dựng PCA thủ công bằng NumPy trên bộ **UCI Human Activity Recognition Using Smartphones** mà **không sử dụng sklearn** cho **StandardScaler/PCA**.

Notebook này triển khai 4 giai đoạn đầu:

1. Chuẩn bị và kiểm tra dữ liệu.
2. Standardization tự xây dựng.
3. Xây dựng **PCAFromScratch**.
4. Kiểm chứng PCA bằng các tính chất toán học.

Quy tắc bài tập

- Không dùng `sklearn.preprocessing.StandardScaler`.
- Không dùng `sklearn.decomposition.PCA`.
- Có thể dùng NumPy cho đại số tuyến tính (`np.mean`, `np.std`, `np.linalg.eigh`, phép nhân ma trận...).
- PCA chỉ được **fit trên tập train** để tránh data leakage.
- `y_train/y_test` không tham gia vào quá trình fit PCA; label chỉ dùng cho kiểm tra/phân tích sau này.

Quy ước AI PROMPTING LOG

Mỗi giai đoạn quan trọng có một **AI PROMPTING LOG**. Prompt được viết theo cách:

- mô tả rõ dữ liệu đầu vào;
- nêu đúng ràng buộc kỹ thuật;
- yêu cầu cụ thể đầu ra;
- nêu các kiểm tra cần có;
- đủ độc lập để copy sang một agent/máy khác và tạo ra code có chức năng tương đương.

Điều này giúp log phản ánh **logic xử lý của giai đoạn**, thay vì chỉ ghi một câu kiểu "hãy viết code PCA".

AI PROMPTING LOG — P00: Thiết kế pipeline tổng thể

Mục tiêu: yêu cầu agent tạo kiến trúc notebook PCA from scratch đúng với bài toán UCI HAR.

Prompt đã sử dụng / có thể tái sử dụng:

Tôi đang làm bài giảm chiều dữ liệu bằng PCA trên bộ UCI Human Activity Recognition Using Smartphones. Dataset có train/test riêng và 561 features. Hãy thiết kế notebook Python chạy được trên Google Colab theo pipeline: load dữ liệu → kiểm tra dữ liệu → chuẩn hóa bằng công thức z-score tự viết → tính covariance matrix → eigendecomposition bằng NumPy → sắp xếp eigenvalues/eigenvectors → tính explained variance ratio → transform dữ liệu sang PCA space → inverse transform → kiểm chứng các tính chất toán học của PCA. Không được dùng sklearn StandardScaler hoặc sklearn PCA. PCA và mọi thống kê preprocessing chỉ được fit trên training set; test set chỉ được transform bằng tham số đã học từ train. Hãy chia code thành các hàm/class rõ ràng, có assertion/checkpoint và output chẩn đoán để phát hiện lỗi.

Đầu ra mong đợi: một pipeline tái lập được, tách train/test đúng và có checkpoint ở từng giai đoạn.

✓ Giai đoạn 1 — Chuẩn bị và kiểm tra dữ liệu

Mục tiêu của giai đoạn này:

- tải đúng UCI HAR;
- đọc `X_train`, `X_test`, `y_train`, `y_test`;
- đọc tên feature và activity label;
- kiểm tra shape, NaN, Inf, nhãn, phân bố lớp;
- phát hiện feature variance gần bằng 0;
- không thay đổi dữ liệu ở bước này.

AI PROMPTING LOG — P01: Load và audit UCI HAR

Prompt:

Viết code Python/Google Colab để chuẩn bị bộ “UCI Human Activity Recognition Using Smartphones” mà không dùng sklearn. Code phải xử lý bền vững ba trường hợp. Thứ nhất, nếu dataset đã được giải nén sẵn thì tự phát hiện thư mục gốc dựa trên `train/X_train.txt`. Thứ hai, nếu cần tải từ UCI thì thử URL chính thức và URL fallback; lưu ý gói UCI có thể là ZIP ngoài chứa tiếp file `UCI HAR Dataset.zip`, vì vậy phải tự phát hiện và giải nén nested ZIP cho đến khi tìm thấy cấu trúc `train/X_train.txt`, `train/y_train.txt`, `test/X_test.txt`, `test/y_test.txt`, `features.txt`, `activity_labels.txt`. Thứ ba, nếu tải tự động thất bại hoặc sau giải nén vẫn không tìm thấy dataset, trên Google Colab hãy gọi `google.colab.files.upload()` để người dùng chọn file ZIP từ máy, rồi tiếp tục tự giải nén và phát hiện dataset. Sau khi tìm thấy dataset, đọc X/y bằng NumPy, metadata bằng Pandas; kiểm tra shape train/test, 561 features, số class, class distribution, NaN, Inf, variance gần 0 và label có khớp số mẫu. Tạo assertion cho các điều kiện quan trọng. Không dùng sklearn.

Logic cần giữ nguyên khi tái tạo: phát hiện dữ liệu local → xử lý ZIP/nested ZIP → thử download → nếu thất bại thì manual upload → tự tìm dataset root → parse → audit → assert.

```
# =====
# 1. Imports và cấu hình
# =====
from pathlib import Path
import urllib.request
import zipfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.set_printoptions(precision=6, suppress=True)

DATA_ROOT = Path("/content") if Path("/content").exists() else Path.cwd()
ZIP_PATH = DATA_ROOT / "UCI_HAR_Dataset.zip"
DATASET_DIR = DATA_ROOT / "UCI HAR Dataset"

# Official UCI static file first; legacy UCI URL is kept as fallback.
UCI_HAR_URLS = [
    "https://archive.ics.uci.edu/static/public/240/"
    "human%2Bactivity%2Brecognition%2Busing%2Bsmartphones.zip",
    "https://archive.ics.uci.edu/ml/machine-learning-databases/00240/"
    "UCI%20HAR%20Dataset.zip",
]

print("Working directory:", DATA_ROOT)
```

Working directory: /content

```
# =====
# 2. Chuẩn bị dataset ROBUST
#
# Hỗ trợ 3 trường hợp:
# A. Dataset đã có sẵn trong /content/UCI HAR Dataset
# B. Tự tải từ UCI, kể cả ZIP lồng ZIP
# C. Nếu tải thất bại: upload file ZIP từ máy lên Colab
# =====

def dataset_is_ready(dataset_dir: Path) -> bool:
    """Kiểm tra đúng cấu trúc tối thiểu của UCI HAR."""
    required = [
        dataset_dir / "train" / "X_train.txt",
        dataset_dir / "train" / "y_train.txt",
        dataset_dir / "test" / "X_test.txt",
        dataset_dir / "test" / "y_test.txt",
        dataset_dir / "features.txt",
        dataset_dir / "activity_labels.txt",
    ]
    return all(p.exists() for p in required)

def extract_zip(zip_path: Path, destination: Path):
    """Giải nén một file ZIP."""
    print("Giải nén:", zip_path)
```

```

with zipfile.ZipFile(zip_path, "r") as zf:
    zf.extractall(destination)

def locate_dataset_root(search_root: Path):
    """
    Tìm thư mục dataset dựa trên file train/X_train.txt,
    sau đó xác nhận đủ cấu trúc UCI HAR.
    """
    for x_train_path in search_root.rglob("train/X_train.txt"):
        candidate_root = x_train_path.parent.parent
        if dataset_is_ready(candidate_root):
            return candidate_root
    return None

def extract_nested_uci_zips(search_root: Path, primary_zip: Path | None = None):
    """
    UCI hiện có thể phát hành:
    outer.zip
    └─ UCI HAR Dataset.names
    └─ UCI HAR Dataset.zip  <-- ZIP bên trong

    Hàm này tìm và giải nén các ZIP liên quan một cách an toàn.
    """
    processed = set()

    # Tối đa vài vòng để tránh vòng lặp vô hạn nếu ZIP lồng.
    for _ in range(4):
        dataset_root = locate_dataset_root(search_root)
        if dataset_root is not None:
            return dataset_root

        zip_candidates = list(search_root.rglob("*.zip"))
        found_new = False

        for zip_file in zip_candidates:
            resolved = str(zip_file.resolve())

            if resolved in processed:
                continue

            # Chỉ xử lý ZIP có liên quan tới UCI HAR,
            # hoặc ZIP chính đã được chỉ định.
            relevant = (
                "UCI HAR" in zip_file.name
                or "UCI_HAR" in zip_file.name
                or (primary_zip is not None and zip_file.resolve() == primary_zip.resolve())
            )

            if not relevant:
                continue

            processed.add(resolved)

            try:
                extract_zip(zip_file, search_root)
                found_new = True
            except zipfile.BadZipFile:
                print("Bỏ qua file không phải ZIP hợp lệ:", zip_file)

            dataset_root = locate_dataset_root(search_root)
            if dataset_root is not None:
                return dataset_root

        if not found_new:
            break

    return locate_dataset_root(search_root)

def upload_zip_from_computer():
    """
    Fallback dành cho Google Colab:
    người dùng chọn file ZIP UCI HAR từ máy.

    Có thể upload:

```

```

- file ngoài tải từ UCI, hoặc
- trực tiếp file 'UCI HAR Dataset.zip'.
"""
try:
    from google.colab import files
except ImportError as exc:
    raise RuntimeError(
        "Không chạy trong Google Colab. "
        "Hãy tự copy file ZIP UCI HAR vào thư mục làm việc rồi chạy lại cell."
    ) from exc

print("\n=== MANUAL UPLOAD FALLBACK ===")
print("Hãy chọn file ZIP UCI HAR từ máy.")
print("Có thể dùng file tải từ UCI hoặc file 'UCI HAR Dataset.zip'.")

uploaded = files.upload()

if not uploaded:
    raise RuntimeError("Không có file nào được upload.")

uploaded_paths = []

for filename, file_bytes in uploaded.items():
    target = DATA_ROOT / filename

    # files.upload thường đã tạo file trong /content,
    # nhưng ghi lại bytes giúp code ổn định hơn.
    with open(target, "wb") as f:
        f.write(file_bytes)

    uploaded_paths.append(target)
    print("Đã upload:", target)

return uploaded_paths

def ensure_uci_har_dataset():
    global DATASET_DIR

    # -----
    # A. Dataset đã tồn tại đúng cấu trúc
    # -----
    existing_root = locate_dataset_root(DATA_ROOT)

    if existing_root is not None:
        DATASET_DIR = existing_root
        print("Dataset đã sẵn sàng:", DATASET_DIR)
        return DATASET_DIR

    # -----
    # B1. Nếu /content đã có ZIP do người dùng upload trước,
    #     thử giải nén trước khi download lại.
    # -----
    local_root = extract_nested_uci_zips(DATA_ROOT)

    if local_root is not None:
        DATASET_DIR = local_root
        print("Dataset tìm thấy từ ZIP local:", DATASET_DIR)
        return DATASET_DIR

    # -----
    # B2. Tự động download từ UCI
    # -----
    download_success = False
    last_error = None

    if not ZIP_PATH.exists():
        print("Đang thử tải UCI HAR tự động...")

        for url in UCI_HAR_URLS:
            try:
                print("Thử nguồn:", url)
                urllib.request.urlretrieve(url, ZIP_PATH)
                print("Đã tải:", ZIP_PATH)
                download_success = True
                last_error = None
                break

```

```

except Exception as exc:
    last_error = exc
    print("Nguồn này thất bại:", repr(exc))

    if ZIP_PATH.exists():
        try:
            ZIP_PATH.unlink()
        except Exception:
            pass
    else:
        print("Đã có ZIP chính:", ZIP_PATH)
        download_success = True

# -----
# B3. Giải nén ZIP tải tự động, kể cả nested ZIP
# -----
if download_success:
    try:
        dataset_root = extract_nested_uci_zips(
            DATA_ROOT,
            primary_zip=ZIP_PATH
        )

        if dataset_root is not None:
            DATASET_DIR = dataset_root
            print("Dataset đã sẵn sàng:", DATASET_DIR)
            return DATASET_DIR

    except Exception as exc:
        print("Tải được file nhưng giải nén/phát hiện dataset thất bại:")
        print(repr(exc))

# -----
# C. FALLBACK: upload file ZIP từ máy
# -----
print("\nKhông thể chuẩn bị dataset hoàn toàn tự động.")
print("Chuyển sang phương án upload dữ liệu từ máy.")

uploaded_paths = upload_zip_from_computer()

# Giải nén file vừa upload, kể cả khi đó là outer ZIP.
for uploaded_path in uploaded_paths:
    if uploaded_path.suffix.lower() == ".zip":
        try:
            extract_zip(uploaded_path, DATA_ROOT)
        except zipfile.BadZipFile:
            print("File upload không phải ZIP hợp lệ:", uploaded_path)

dataset_root = extract_nested_uci_zips(DATA_ROOT)

if dataset_root is None:
    visible_items = sorted(p.name for p in DATA_ROOT.iterdir())
    raise RuntimeError(
        "Đã upload nhưng vẫn chưa tìm thấy cấu trúc UCI HAR hợp lệ.\n"
        "Hãy upload file ZIP chính thức chứa thư mục train/test.\n"
        f"Các mục hiện có trong {DATA_ROOT}: {visible_items}"
    )

DATASET_DIR = dataset_root
print("Dataset đã sẵn sàng sau manual upload:", DATASET_DIR)
return DATASET_DIR

DATASET_DIR = ensure_uci_har_dataset()

print("\nDATASET_DIR cuối cùng:", DATASET_DIR)

Dataset đã sẵn sàng: /content/UCI HAR Dataset

DATASET_DIR cuối cùng: /content/UCI HAR Dataset

```

```

# =====
# 3. Hàm đọc dataset
# =====
def load_uci_har(dataset_dir: Path):
    train_dir = dataset_dir / "train"

```

```

test_dir = dataset_dir / "test"

X_train = np.loadtxt(train_dir / "X_train.txt", dtype=np.float64)
y_train = np.loadtxt(train_dir / "y_train.txt", dtype=np.int64)
X_test = np.loadtxt(test_dir / "X_test.txt", dtype=np.float64)
y_test = np.loadtxt(test_dir / "y_test.txt", dtype=np.int64)

features_df = pd.read_csv(
    dataset_dir / "features.txt",
    sep=r"\s+",
    header=None,
    names=["index", "feature"]
)
activity_df = pd.read_csv(
    dataset_dir / "activity_labels.txt",
    sep=r"\s+",
    header=None,
    names=["label", "activity"]
)

feature_names = features_df["feature"].astype(str).tolist()
activity_map = dict(zip(activity_df["label"], activity_df["activity"]))

return X_train, X_test, y_train, y_test, feature_names, activity_map

X_train, X_test, y_train, y_test, feature_names, activity_map = load_uci_har(DATASET_DIR)

print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
print("Số feature names:", len(feature_names))
print("Activities:", activity_map)

X_train: (7352, 561)
X_test : (2947, 561)
y_train: (7352,)
y_test : (2947,)
Số feature names: 561
Activities: {1: 'WALKING', 2: 'WALKING_UPSTAIRS', 3: 'WALKING_DOWNSTAIRS', 4: 'SITTING', 5: 'STANDING', 6: 'LAYING'}

```

```

# =====
# 4. Data audit / checkpoint 1
# =====
def audit_dataset(X_train, X_test, y_train, y_test, feature_names, activity_map):
    print("=== DATASET AUDIT ===")
    print(f"Train samples : {X_train.shape[0]}")
    print(f"Test samples  : {X_test.shape[0]}")
    print(f"Features       : {X_train.shape[1]}")
    print(f"Classes        : {len(np.unique(y_train))}")

    print("\nNaN / Inf:")
    print("X_train NaN:", np.isnan(X_train).sum())
    print("X_train Inf:", np.isinf(X_train).sum())
    print("X_test  NaN:", np.isnan(X_test).sum())
    print("X_test  Inf:", np.isinf(X_test).sum())

    train_var = np.var(X_train, axis=0)
    near_zero_mask = train_var < 1e-12
    print("\nFeature variance:")
    print("Min variance:", train_var.min())
    print("Max variance:", train_var.max())
    print("Số feature variance < 1e-12:", int(near_zero_mask.sum()))

    print("\nClass distribution - train:")
    labels, counts = np.unique(y_train, return_counts=True)
    for label, count in zip(labels, counts):
        print(f"{label}: {activity_map.get(int(label), 'UNKNOWN')}<20 {count}")

    print("\nClass distribution - test:")
    labels, counts = np.unique(y_test, return_counts=True)
    for label, count in zip(labels, counts):
        print(f"{label}: {activity_map.get(int(label), 'UNKNOWN')}<20 {count}")

    # Assertions
    assert X_train.ndim == 2 and X_test.ndim == 2

```

```

assert X_train.shape[1] == X_test.shape[1] == 561
assert len(feature_names) == 561
assert X_train.shape[0] == len(y_train)
assert X_test.shape[0] == len(y_test)
assert np.isfinite(X_train).all()
assert np.isfinite(X_test).all()
assert set(np.unique(y_train)).issubset(set(activity_map.keys()))
assert set(np.unique(y_test)).issubset(set(activity_map.keys()))

print("\nCHECKPOINT 1: PASS ✅")

audit_dataset(
    X_train, X_test, y_train, y_test,
    feature_names, activity_map
)

```

=== DATASET AUDIT ===

Train samples : 7352
 Test samples : 2947
 Features : 561
 Classes : 6

NaN / Inf:

X_train NaN: 0
 X_train Inf: 0
 X_test NaN: 0
 X_test Inf: 0

Feature variance:

Min variance: 0.0016652723790956574
 Max variance: 0.5650254817385447
 Số feature variance < 1e-12: 0

Class distribution – train:

1: WALKING	1226
2: WALKING_UPSTAIRS	1073
3: WALKING_DOWNSTAIRS	986
4: SITTING	1286
5: STANDING	1374
6: LAYING	1407

Class distribution – test:

1: WALKING	496
2: WALKING_UPSTAIRS	471
3: WALKING_DOWNSTAIRS	420
4: SITTING	491
5: STANDING	532
6: LAYING	537

CHECKPOINT 1: PASS ✅

✎ Giai đoạn 2 — Standardization tự xây dựng

PCA nhạy với scale. Vì vậy ta chuẩn hóa từng feature:

$$[z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}]$$

Quan trọng: `mean` và `std` chỉ được tính từ `X_train`.

Test set được transform bằng chính `mean/std` của train:

$$[X_{test}^{scaled}]$$

$$\frac{x_{test} - \mu_{train}}{\sigma_{train}}$$

Việc này tránh **data leakage**.

✎ AI PROMPTING LOG — P02: Manual Standardization không leakage

Prompt:

Với các biến NumPy `X_train` và `X_test` của UCI HAR, hãy tự xây dựng standardization theo z-score mà không dùng sklearn. Viết một class hoặc các hàm có `fit`, `transform`, `fit_transform`. `fit` chỉ được gọi trên training set và phải lưu `mean_` cùng `scale_`; nếu std của feature bằng hoặc gần 0 thì thay bằng 1 để tránh chia 0. Sau đó transform train và test

bằng cùng tham số. Hãy kiểm tra training data sau scale có mean gần 0 và std gần 1 đối với các feature có variance khác 0; kiểm tra không có NaN/Inf. In max absolute mean, min/max std sau chuẩn hóa và tạo checkpoint PASS/FAIL. Không dùng sklearn.

Logic cần giữ nguyên: fit train statistics → protect zero std → transform train/test → numerical validation.

```
# =====
# 5. Manual Standardizer
# =====
class StandardizerFromScratch:
    def __init__(self, eps=1e-12):
        self.eps = eps
        self.mean_ = None
        self.scale_ = None
        self.zero_variance_mask_ = None
        self.is_fitted_ = False

    def fit(self, X):
        X = np.asarray(X, dtype=np.float64)

        self.mean_ = np.mean(X, axis=0)
        raw_std = np.std(X, axis=0, ddof=0)

        self.zero_variance_mask_ = raw_std < self.eps
        self.scale_ = raw_std.copy()
        self.scale_[self.zero_variance_mask_] = 1.0

        self.is_fitted_ = True
        return self

    def transform(self, X):
        if not self.is_fitted_:
            raise RuntimeError("Standardizer chưa được fit.")

        X = np.asarray(X, dtype=np.float64)
        return (X - self.mean_) / self.scale_

    def fit_transform(self, X):
        return self.fit(X).transform(X)

scaler = StandardizerFromScratch(eps=1e-12)

# Chỉ fit trên TRAIN
X_train_scaled = scaler.fit_transform(X_train)

# TEST chỉ transform bằng statistics học từ TRAIN
X_test_scaled = scaler.transform(X_test)

print("X_train_scaled:", X_train_scaled.shape)
print("X_test_scaled :", X_test_scaled.shape)

X_train_scaled: (7352, 561)
X_test_scaled : (2947, 561)

# =====
# 6. Kiểm tra standardization / checkpoint 2
# =====
scaled_mean = np.mean(X_train_scaled, axis=0)
scaled_std = np.std(X_train_scaled, axis=0, ddof=0)

non_constant = ~scaler.zero_variance_mask_

max_abs_mean = np.max(np.abs(scaled_mean))
max_std_error = np.max(np.abs(scaled_std[non_constant] - 1.0)) if np.any(non_constant) else 0.0

print("=== STANDARDIZATION CHECK ===")
print("Max |mean| trên train scaled:", max_abs_mean)
print("Max |std - 1| (non-constant features):", max_std_error)
print("Zero/near-zero variance features:", int(scaler.zero_variance_mask_.sum()))
print("Train finite:", np.isfinite(X_train_scaled).all())
print("Test finite :", np.isfinite(X_test_scaled).all())

assert np.isfinite(X_train_scaled).all()
assert np.isfinite(X_test_scaled).all()
```

```

assert max_abs_mean < 1e-10
assert max_std_error < 1e-10

print("\nCHECKPOINT 2: PASS ✅")

=== STANDARDIZATION CHECK ===
Max |mean| trên train scaled: 3.1128051525275424e-13
Max |std - 1| (non-constant features): 5.440092820663267e-14
Zero/near-zero variance features: 0
Train finite: True
Test finite : True

CHECKPOINT 2: PASS ✅

```

▼ Giai đoạn 3 — Xây dựng PCA From Scratch

Ta triển khai PCA theo đúng cốt lõi toán học:

1. dữ liệu đã chuẩn hóa;
2. tính covariance matrix;
3. eigendecomposition;
4. sort eigenvalue giảm dần;
5. tính explained variance ratio;
6. chọn eigenvectors đầu tiên làm principal axes;
7. project dữ liệu: $Z = XW_k$
8. reconstruct: $\hat{X} = ZW_k^T$

Vì covariance matrix là ma trận đối xứng, dùng `np.linalg.eigh` thay cho `np.linalg.eig`.

▼ AI PROMPTING LOG — P03: PCAFromScratch bằng covariance + eigendecomposition

Prompt:

Viết class `PCAFromScratch` bằng NumPy, không dùng sklearn. Input là dữ liệu đã standardize. Trong `fit(X)`: tính covariance matrix thủ công theo $(X.T @ X) / (n_samples - 1)$; dùng `np.linalg.eigh` vì covariance matrix đối xứng; sắp xếp eigenvalues giảm dần và reorder eigenvectors tương ứng; loại sai số eigenvalue âm rất nhỏ do floating point nếu cần; tính `explained_variance_ratio_` và `cumulative_explained_variance_`; lưu `components_` theo convention mỗi cột là một eigenvector. Hỗ trợ `n_components=None` hoặc một số nguyên hợp lệ. Trong `transform(X)`, project bằng $X @ components_$; trong `inverse_transform(Z)` reconstruct bằng $Z @ components_.T$; thêm `fit_transform`. Hãy validate kích thước và không được dùng label y. Code phải đủ tổng quát để chạy với 561 features của UCI HAR.

Logic cần giữ nguyên: covariance → `eigh` → descending sort → EVR → choose (W_k) → transform/inverse-transform.

```

# =====
# 7. PCA From Scratch
# =====
class PCAFromScratch:
    def __init__(self, n_components=None, negative_eigen_tol=1e-10):
        self.n_components = n_components
        self.negative_eigen_tol = negative_eigen_tol

        self.n_features_in_ = None
        self.n_components_ = None

        self.covariance_ = None
        self.eigenvalues_all_ = None
        self.eigenvectors_all_ = None

        self.explained_variance_ = None
        self.explained_variance_ratio_ = None
        self.cumulative_explained_variance_ = None

        self.components_ = None
        self.is_fitted_ = False

    def fit(self, X):
        X = np.asarray(X, dtype=np.float64)

```

```

if X.ndim != 2:
    raise ValueError("X phải là ma trận 2D.")

n_samples, n_features = X.shape
if n_samples < 2:
    raise ValueError("Cần ít nhất 2 samples để tính covariance.")

self.n_features_in_ = n_features

# 1) Covariance matrix
self.covariance_ = (X.T @ X) / (n_samples - 1)

# 2) Eigendecomposition cho ma trận đối xứng
eigenvalues, eigenvectors = np.linalg.eigh(self.covariance_)

# 3) Sort giảm dần
order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order]
eigenvectors = eigenvectors[:, order]

# 4) Xử lý sai số floating point rất nhỏ
if np.min(eigenvalues) < -self.negative_eigen_tol:
    raise ValueError(
        "Phát hiện eigenvalue âm đáng kể; covariance/PCA có thể có lỗi."
    )

eigenvalues = np.where(
    (eigenvalues < 0) & (np.abs(eigenvalues) <= self.negative_eigen_tol),
    0.0,
    eigenvalues
)

self.eigenvalues_all_ = eigenvalues
self.eigenvectors_all_ = eigenvectors

total_variance = np.sum(eigenvalues)
if total_variance <= 0:
    raise ValueError("Tổng variance không dương.")

explained_ratio_all = eigenvalues / total_variance
cumulative_all = np.cumsum(explained_ratio_all)

# 5) Resolve n_components
if self.n_components is None:
    k = n_features
else:
    k = int(self.n_components)
    if not (1 <= k <= n_features):
        raise ValueError(
            f"n_components phải nằm trong [1, {n_features}]."
        )

self.n_components_ = k

# Convention: columns = principal axes
self.components_ = eigenvectors[:, :k]
self.explained_variance_ = eigenvalues[:k]
self.explained_variance_ratio_ = explained_ratio_all[:k]
self.cumulative_explained_variance_ = cumulative_all[:k]

self.is_fitted_ = True
return self

def transform(self, X):
    if not self.is_fitted_:
        raise RuntimeError("PCA chưa được fit.")

    X = np.asarray(X, dtype=np.float64)

    if X.ndim != 2 or X.shape[1] != self.n_features_in_:
        raise ValueError(
            f"X phải có shape (n_samples, {self.n_features_in_})."
        )

    return X @ self.components_

def fit_transform(self, X):

```

```

return self.fit(X).transform(X)

def inverse_transform(self, Z):
    if not self.is_fitted_:
        raise RuntimeError("PCA chưa được fit.")

    Z = np.asarray(Z, dtype=np.float64)

    if Z.ndim != 2 or Z.shape[1] != self.n_components_:
        raise ValueError(
            f"Z phải có shape (n_samples, {self.n_components_})."
        )

    return Z @ self.components_.T

# Fit FULL PCA trên TRAIN để phục vụ phân tích variance.
pca_full = PCAFromScratch(n_components=None)
pca_full.fit(X_train_scaled)

print("Covariance shape:", pca_full.covariance_.shape)
print("Eigenvalues shape:", pca_full.eigenvalues_all_.shape)
print("Eigenvectors shape:", pca_full.eigenvectors_all_.shape)
print("Components retained:", pca_full.n_components_)

Covariance shape: (561, 561)
Eigenvalues shape: (561,)
Eigenvectors shape: (561, 561)
Components retained: 561

```

✧ Giai đoạn 4 — Kiểm chứng PCA hoạt động đúng

Ta **không dùng sklearn làm oracle**. Thay vào đó kiểm chứng trực tiếp các tính chất toán học:

1. Covariance matrix đối xứng.
2. Eigenvalues không âm (cho phép sai số rất nhỏ).
3. Eigenvectors trực giao.
4. Tổng explained variance ratio ≈ 1 .
5. Eigenvalues/variance được sắp xếp giảm dần.
6. Covariance của PCA scores gần đường chéo.
7. Variance của từng PC gần bằng eigenvalue tương ứng.

Nếu các kiểm tra này đạt, implementation PCA được xem là đúng về mặt số học/toán học.

✧ AI PROMPTING LOG — P04: Mathematical validation cho PCA tự xây dựng

Prompt:

Tôi đã có class PCA tự xây dựng bằng covariance matrix và `np.linalg.eigh`, đã fit trên `X_train_scaled`. Hãy viết bộ validation không dựa vào sklearn gồm: (1) kiểm tra covariance matrix đối xứng bằng max absolute symmetry error; (2) kiểm tra eigenvalues không âm ngoài tolerance; (3) kiểm tra trực giao eigenvectors qua `V.T @ V` so với identity; (4) kiểm tra tổng explained variance ratio gần 1; (5) kiểm tra eigenvalues đã giảm dần; (6) transform toàn bộ train bằng tất cả PCs rồi tính covariance của PCA scores, đo max off-diagonal absolute value để xác nhận các PCs gần không tương quan; (7) so sánh diagonal của score covariance với eigenvalues. In từng metric và assertion với tolerance hợp lý. Không dùng sklearn.

Logic cần giữ nguyên: kiểm chứng từ các invariant toán học của PCA, không so sánh với thư viện PCA có sẵn.

```

# =====
# 8. Mathematical validation / checkpoint 3
# =====
def validate_pca_math(pca, X_scaled, tol=1e-8):
    print("=== PCA MATHEMATICAL VALIDATION ===")

    C = pca.covariance_
    V = pca.eigenvectors_all_
    lambdas = pca.eigenvalues_all_

    # 1. Covariance symmetry
    symmetry_error = np.max(np.abs(C - C.T))

```

```

print("1) Max covariance symmetry error:", symmetry_error)

# 2. Eigenvalue non-negativity
min_eigenvalue = np.min(lambdas)
print("2) Minimum eigenvalue:", min_eigenvalue)

# 3. Orthogonality
I = np.eye(V.shape[1])
orthogonality_error = np.max(np.abs(V.T @ V - I))
print("3) Max eigenvector orthogonality error:", orthogonality_error)

# 4. Sum explained variance ratio
evr_all = lambdas / np.sum(lambdas)
evr_sum = np.sum(evr_all)
print("4) Sum explained variance ratio:", evr_sum)

# 5. Descending order
max_order_violation = np.max(np.diff(lambdas))
sorted_descending = np.all(np.diff(lambdas) <= tol)
print("5) Eigenvalues descending:", sorted_descending)
print("    Max order violation:", max_order_violation)

# 6. Transform bằng toàn bộ PCs
Z_full = X_scaled @ V
cov_Z = (Z_full.T @ Z_full) / (Z_full.shape[0] - 1)

off_diag = cov_Z - np.diag(np.diag(cov_Z))
max_off_diag = np.max(np.abs(off_diag))
print("6) Max |off-diagonal| of PCA-score covariance:", max_off_diag)

# 7. Diagonal should match eigenvalues
diag_error = np.max(np.abs(np.diag(cov_Z) - lambdas))
print("7) Max |diag(cov_Z) - eigenvalue|:", diag_error)

# Assertions
assert symmetry_error < tol
assert min_eigenvalue >= -tol
assert orthogonality_error < tol
assert abs(evr_sum - 1.0) < tol
assert sorted_descending
assert max_off_diag < 1e-7
assert diag_error < 1e-7

metrics = {
    "symmetry_error": symmetry_error,
    "min_eigenvalue": min_eigenvalue,
    "orthogonality_error": orthogonality_error,
    "evr_sum": evr_sum,
    "max_off_diagonal_score_cov": max_off_diag,
    "score_cov_diag_vs_eigen_error": diag_error,
}

print("\nCHECKPOINT 3: PASS ✅")
return metrics

validation_metrics = validate_pca_math(
    pca_full,
    X_train_scaled
)

```

```

=== PCA MATHEMATICAL VALIDATION ===
1) Max covariance symmetry error: 0.0
2) Minimum eigenvalue: 0.0
3) Max eigenvector orthogonality error: 3.219646771412954e-15
4) Sum explained variance ratio: 0.9999999999999998
5) Eigenvalues descending: True
    Max order violation: 0.0
6) Max |off-diagonal| of PCA-score covariance: 1.7281573623722512e-13
7) Max |diag(cov_Z) - eigenvalue|: 4.263256414560601e-14

```

CHECKPOINT 3: PASS ✅

✓ AI PROMPTING LOG — P05: Kiểm tra transform và inverse-transform

Prompt:

Dùng class `PCAFromScratch` đã xây dựng để kiểm tra API transform/inverse_transform trên UCI HAR. Chọn thử một giá trị k nhỏ như 10 chỉ để kiểm tra chức năng, fit PCA trên `X_train_scaled`, transform cả train và test, kiểm tra shape đầu ra, reconstruct train bằng inverse_transform và tính reconstruction MSE. Không dùng label để fit PCA. In shape và MSE; thêm assertion để chắc chắn số cột sau transform bằng k, số sample không đổi, và MSE hữu hạn. Đây chỉ là functional test, chưa dùng k=10 để kết luận k tối ưu.

Logic cần giữ nguyên: test API và shape trước khi bước vào phân tích lựa chọn số components.

```
# =====
# 9. Functional test transform / inverse_transform
# =====
k_test = 10

pca_test = PCAFromScratch(n_components=k_test)
X_train_pca_test = pca_test.fit_transform(X_train_scaled)
X_test_pca_test = pca_test.transform(X_test_scaled)

X_train_reconstructed_test = pca_test.inverse_transform(X_train_pca_test)
reconstruction_mse_test = np.mean(
    (X_train_scaled - X_train_reconstructed_test) ** 2
)

print("=== PCA FUNCTIONAL TEST ===")
print("k_test:", k_test)
print("Train original:", X_train_scaled.shape)
print("Train PCA      :", X_train_pca_test.shape)
print("Test original :", X_test_scaled.shape)
print("Test PCA      :", X_test_pca_test.shape)
print("Reconstruction MSE (k=10):", reconstruction_mse_test)

assert X_train_pca_test.shape == (X_train_scaled.shape[0], k_test)
assert X_test_pca_test.shape == (X_test_scaled.shape[0], k_test)
assert X_train_reconstructed_test.shape == X_train_scaled.shape
assert np.isfinite(reconstruction_mse_test)

print("\nFUNCTIONAL TEST: PASS ✅")
```

```
=== PCA FUNCTIONAL TEST ===
k_test: 10
Train original: (7352, 561)
Train PCA      : (7352, 10)
Test original  : (2947, 561)
Test PCA       : (2947, 10)
Reconstruction MSE (k=10): 0.29184431241309244

FUNCTIONAL TEST: PASS ✅
```

Tổng kết Giai đoạn 1–4

Sau khi chạy thành công đến đây, ta đã có:

- dữ liệu UCI HAR được kiểm tra;
- train/test được giữ tách biệt;
- standardization tự xây dựng, fit **chỉ trên train**;
- PCA tự xây dựng bằng covariance + eigendecomposition;
- `fit`, `transform`, `fit_transform`, `inverse_transform`;
- bộ kiểm chứng toán học độc lập với sklearn;
- functional test của transform/reconstruction;
- AI PROMPTING LOG đủ chi tiết để tái tạo từng khối xử lý.

Điều kiện để chuyển sang Giai đoạn 5

Các dòng sau phải xuất hiện:

```
CHECKPOINT 1: PASS
CHECKPOINT 2: PASS
CHECKPOINT 3: PASS
FUNCTIONAL TEST: PASS
```

Giai đoạn tiếp theo

Sau checkpoint này mới thực hiện:

- Scree Plot;
- Explained Variance Ratio;
- Cumulative Explained Variance;
- tự tìm `k80`, `k90`, `k95`, `k99`;
- 2D/3D PCA visualization;
- reconstruction error theo nhiều `k`;
- loading analysis;
- lựa chọn số PC cuối cùng.

Không chọn `n_components` tối ưu trước khi chạy các phân tích trên.

✧ Giai đoạn 5 — Phân tích Explained Variance và chọn ngưỡng

Ở giai đoạn này, **không fit lại PCA** và không chọn `k` tùy ý.

Ta sử dụng eigenvalues đã học từ `X_train_scaled` trong `pca_full` để:

1. tính Explained Variance Ratio (EVR);
2. tính Cumulative Explained Variance;
3. vẽ Scree Plot;
4. tìm số Principal Components tối thiểu đạt 80%, 90%, 95%, 99%.

Các ngưỡng này là **candidate**, chưa phải kết luận cuối cùng.

✧ AI PROMPTING LOG — P06: Explained Variance + threshold selection

Prompt:

Tôi đã có `pca_full` là PCA from scratch fit trên `X_train_scaled`, trong đó `eigenvalues_all` được sắp xếp giảm dần. Hãy viết code NumPy/Matplotlib, không dùng sklearn, để tính `explained_variance_ratio_all = eigenvalues / sum(eigenvalues)` và `cumulative_variance_all = cumsum(explained_variance_ratio_all)`. Vẽ một Scree Plot của explained variance ratio theo principal component và một biểu đồ cumulative explained variance riêng biệt. Tự tìm số component tối thiểu đạt 80%, 90%, 95%, 99% bằng NumPy, lưu thành dictionary `k_by_threshold`, và hiển thị bảng gồm threshold, số components, variance thực tế giữ lại và phần trăm số chiều được giảm từ 561 features. Thêm assertion rằng cumulative variance không giảm, giá trị cuối gần 1, và các `k` tăng theo threshold. Không dùng label và không fit PCA trên test.

Logic cần giữ nguyên: eigenvalues train → EVR → cumulative EVR → threshold search → compression summary → validation.

```
# =====
# 10. Explained Variance Analysis
# =====
eigenvalues_all = pca_full.eigenvalues_all_

explained_variance_ratio_all = (
    eigenvalues_all / np.sum(eigenvalues_all)
)
cumulative_variance_all = np.cumsum(
    explained_variance_ratio_all
)

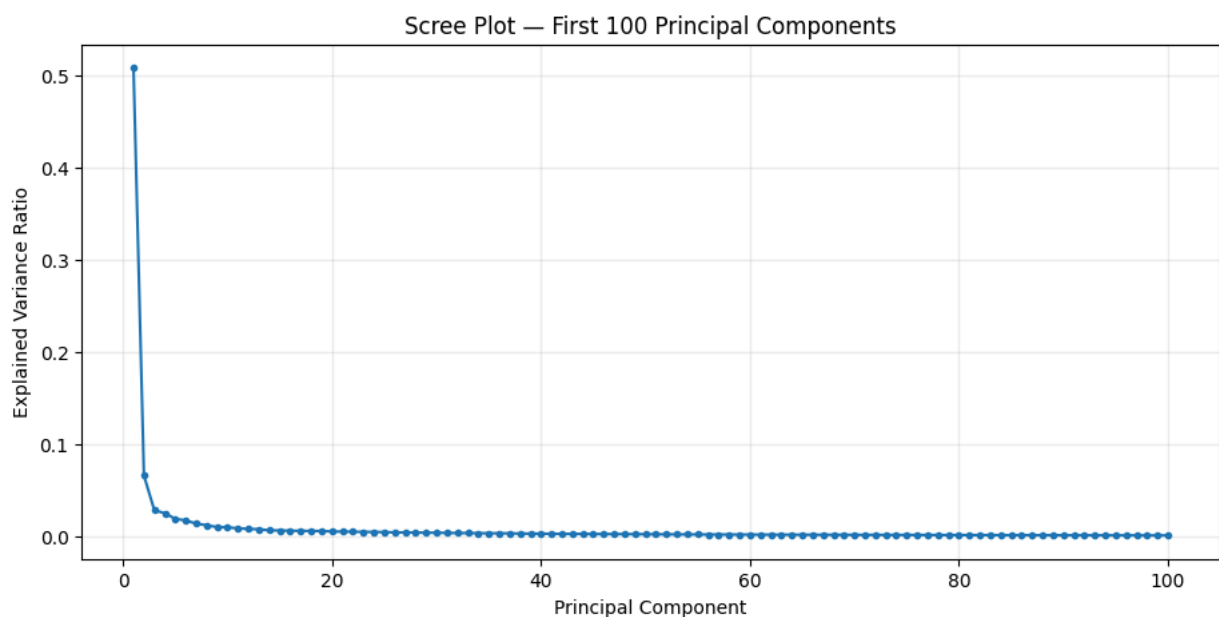
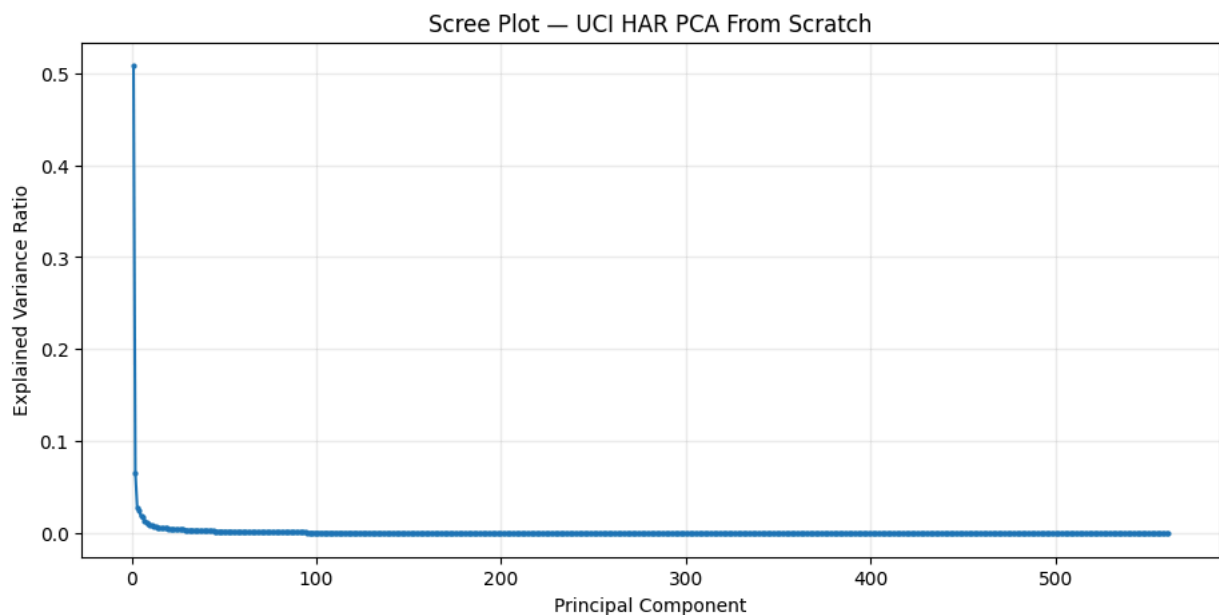
print("Số principal components:", len(eigenvalues_all))
print("Tổng EVR:", explained_variance_ratio_all.sum())
print("Cumulative variance cuối:", cumulative_variance_all[-1])

assert np.all(explained_variance_ratio_all >= -1e-12)
assert np.all(np.diff(cumulative_variance_all) >= -1e-12)
assert abs(cumulative_variance_all[-1] - 1.0) < 1e-10

print("Explained variance checks: PASS ✅")
```

Số principal components: 561
Tổng EVR: 0.9999999999999998
Cumulative variance cuối: 1.000000000000001
Explained variance checks: PASS 

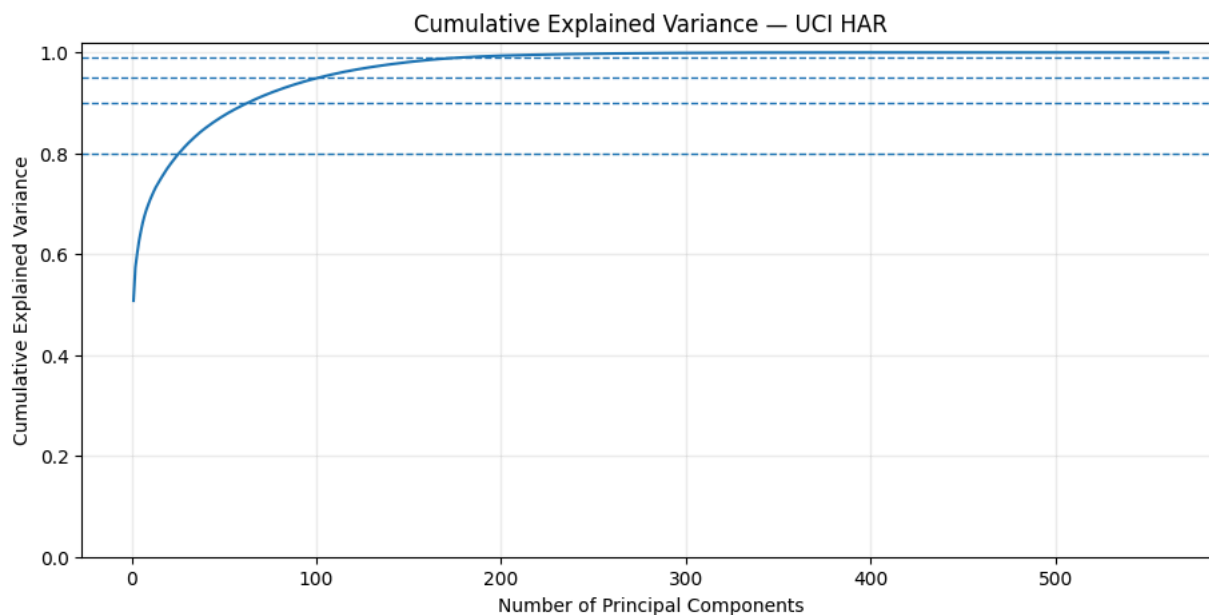
```
# =====  
# 11. Scree Plot  
# =====  
pc_indices = np.arange(1, len(explained_variance_ratio_all) + 1)  
  
plt.figure(figsize=(11, 5))  
plt.plot(pc_indices, explained_variance_ratio_all, marker="o", markersize=2)  
plt.xlabel("Principal Component")  
plt.ylabel("Explained Variance Ratio")  
plt.title("Scree Plot - UCI HAR PCA From Scratch")  
plt.grid(alpha=0.25)  
plt.show()  
  
# Zoom 100 PCs đầu để dễ quan sát elbow.  
n_zoom = min(100, len(pc_indices))  
  
plt.figure(figsize=(11, 5))  
plt.plot(  
    pc_indices[:n_zoom],  
    explained_variance_ratio_all[:n_zoom],  
    marker="o",  
    markersize=3  
)  
plt.xlabel("Principal Component")  
plt.ylabel("Explained Variance Ratio")  
plt.title(f"Scree Plot - First {n_zoom} Principal Components")  
plt.grid(alpha=0.25)  
plt.show()
```



```
# =====
# 12. Cumulative Explained Variance
# =====
plt.figure(figsize=(11, 5))
plt.plot(pc_indices, cumulative_variance_all)
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("Cumulative Explained Variance — UCI HAR")
plt.ylim(0, 1.02)
plt.grid(alpha=0.25)

for threshold in [0.80, 0.90, 0.95, 0.99]:
    plt.axhline(threshold, linestyle="--", linewidth=1)

plt.show()
```



```
# =====
# 13. Try to find k80 / k90 / k95 / k99
# =====
thresholds = [0.80, 0.90, 0.95, 0.99]

k_by_threshold = {}
threshold_rows = []

n_original_features = X_train_scaled.shape[1]

for threshold in thresholds:
    k = int(
        np.searchsorted(
            cumulative_variance_all,
            threshold,
            side="left"
        ) + 1
    )

    retained = float(cumulative_variance_all[k - 1])
    reduction_ratio = 1.0 - (k / n_original_features)

    k_by_threshold[threshold] = k

    threshold_rows.append({
        "Target variance": threshold,
        "Components (k)": k,
        "Actual retained variance": retained,
        "Dimensions reduced": n_original_features - k,
        "Reduction ratio": reduction_ratio,
    })

threshold_df = pd.DataFrame(threshold_rows)

display(threshold_df)

print("\nSelected candidate k values:")
for threshold, k in k_by_threshold.items():
    print(f"{int(threshold*100)}% variance -> k = {k}")

k_values_threshold = list(k_by_threshold.values())
assert all(
    k_values_threshold[i] <= k_values_threshold[i + 1]
    for i in range(len(k_values_threshold) - 1)
)

print("\nCHECKPOINT 4 - Explained Variance: PASS ✅")
```

	Target variance	Components (k)	Actual retained variance	Dimensions reduced	Reduction ratio	
0	0.80	26	0.801805	535	0.953654	
1	0.90	63	0.900535	498	0.887701	
2	0.95	102	0.950756	459	0.818182	
3	0.99	179	0.990193	382	0.680927	

Selected candidate k values:

80% variance -> k = 26

90% variance -> k = 63

95% variance -> k = 102

99% variance -> k = 179

CHECKPOINT 4 – Explained Variance: PASS 

Next steps: [Generate code with threshold_df](#) [New interactive sheet](#)

✧ Giai đoạn 6 — Visualization không gian PCA

Mục tiêu:

- quan sát cấu trúc dữ liệu sau projection;
- xem 6 activity phân bố như thế nào;
- chỉ dùng label để **tô nhóm trên biểu đồ**, không dùng label trong quá trình fit PCA.

Ta trực quan:

- PC1 vs PC2;
- PC1 vs PC3;
- PC2 vs PC3;
- PC1-PC2-PC3 3D.

Lưu ý: overlap trong 2D/3D **không đồng nghĩa PCA thất bại**, vì phần thông tin phân biệt lớp có thể nằm ở các PC cao hơn.

✧ AI PROMPTING LOG — P07: PCA 2D/3D visualization

Prompt:

Dùng eigenvectors của `pca_full` đã fit trên training data để project `X_train_scaled` sang ít nhất 3 principal components bằng phép nhân ma trận NumPy, không dùng sklearn. Viết các biểu đồ Matplotlib riêng biệt cho PC1-vs-PC2, PC1-vs-PC3, PC2-vs-PC3 và một scatter 3D PC1-PC2-PC3. Dùng `y_train` và `activity_map` chỉ để nhóm các điểm khi visualize, tuyệt đối không dùng label trong fit PCA. Mỗi activity phải có legend rõ ràng. Có thể giảm alpha và kích thước điểm để xử lý overlap. Thêm kiểm tra shape và tính variance thực tế của PC1, PC2, PC3 để xác nhận thứ tự variance giảm dần.

Logic cần giữ nguyên: train PCA scores → first 3 PCs → label only for plotting → inspect separability/overlap → variance sanity check.

```
# =====
# 14. PCA scores cho visualization
# =====
V_all = pca_full.eigenvectors_all_

Z_train_full = X_train_scaled @ V_all
Z_train_3d = Z_train_full[:, :3]

print("Z_train_3d shape:", Z_train_3d.shape)

pc_variances = np.var(Z_train_3d, axis=0, ddof=1)
print("Variance PC1-PC3:", pc_variances)

assert Z_train_3d.shape == (X_train_scaled.shape[0], 3)
assert pc_variances[0] >= pc_variances[1] - 1e-10
assert pc_variances[1] >= pc_variances[2] - 1e-10

print("PCA visualization preparation: PASS )
```

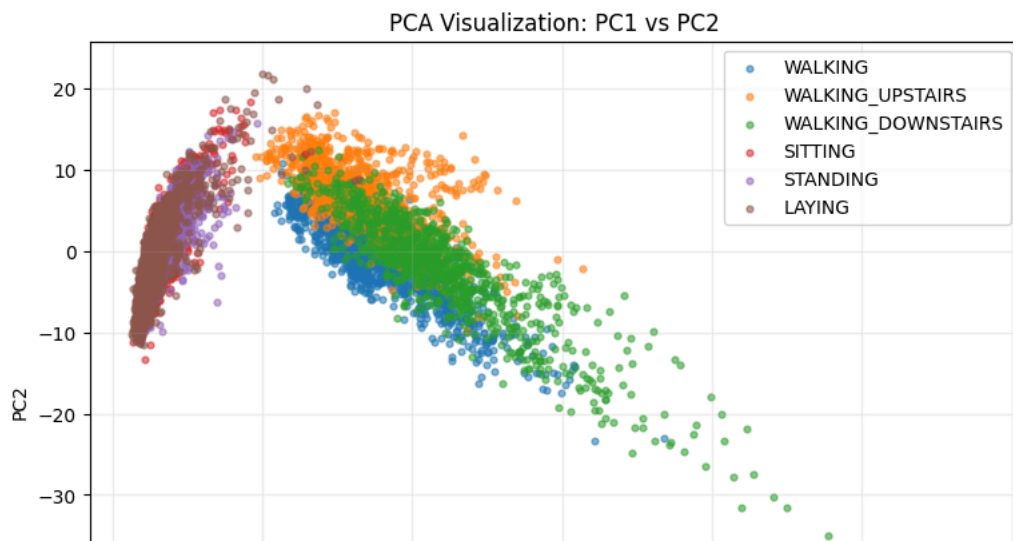
```
Z_train_3d shape: (7352, 3)
Variance PC1-PC3: [284.921131  36.922638  15.746252]
PCA visualization preparation: PASS 
```

```
# =====
# 15. Helper cho scatter 2D
# =====
def plot_pca_2d(Z, y, activity_map, pc_x, pc_y):
    plt.figure(figsize=(9, 7))

    for label in np.unique(y):
        mask = y == label
        plt.scatter(
            Z[mask, pc_x],
            Z[mask, pc_y],
            s=12,
            alpha=0.55,
            label=activity_map.get(int(label), str(label))
        )

    plt.xlabel(f"PC{pc_x + 1}")
    plt.ylabel(f"PC{pc_y + 1}")
    plt.title(f"PCA Visualization: PC{pc_x + 1} vs PC{pc_y + 1}")
    plt.legend()
    plt.grid(alpha=0.2)
    plt.show()

plot_pca_2d(Z_train_3d, y_train, activity_map, 0, 1)
plot_pca_2d(Z_train_3d, y_train, activity_map, 0, 2)
plot_pca_2d(Z_train_3d, y_train, activity_map, 1, 2)
```

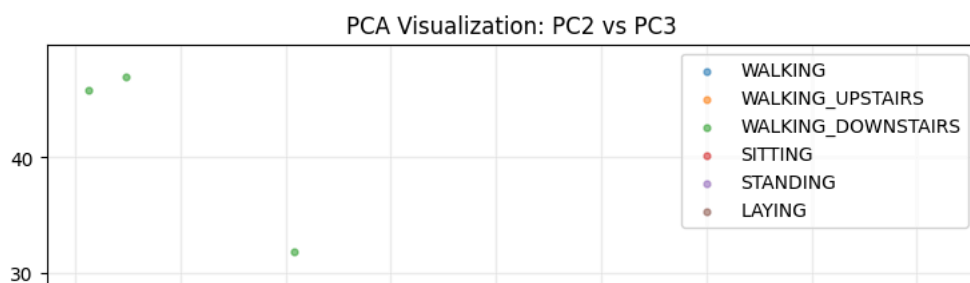
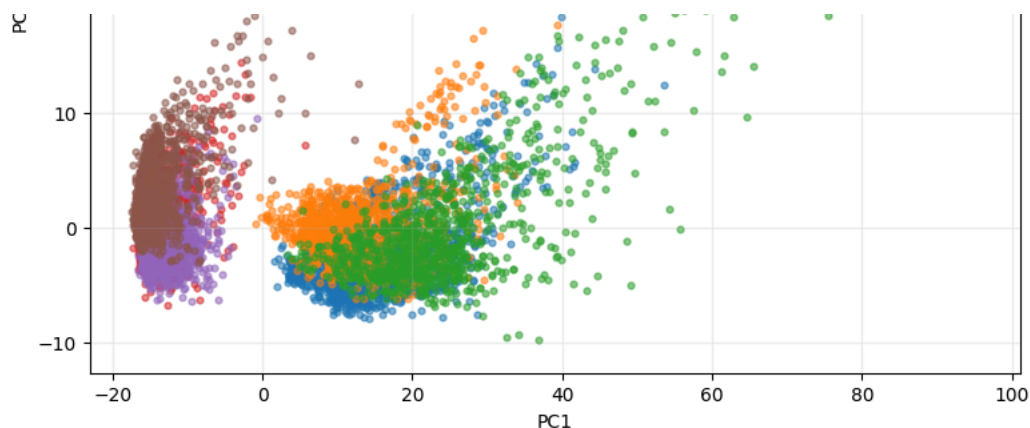
```
# =====
# 16. PCA 3D Visualization
# =====
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection="3d")

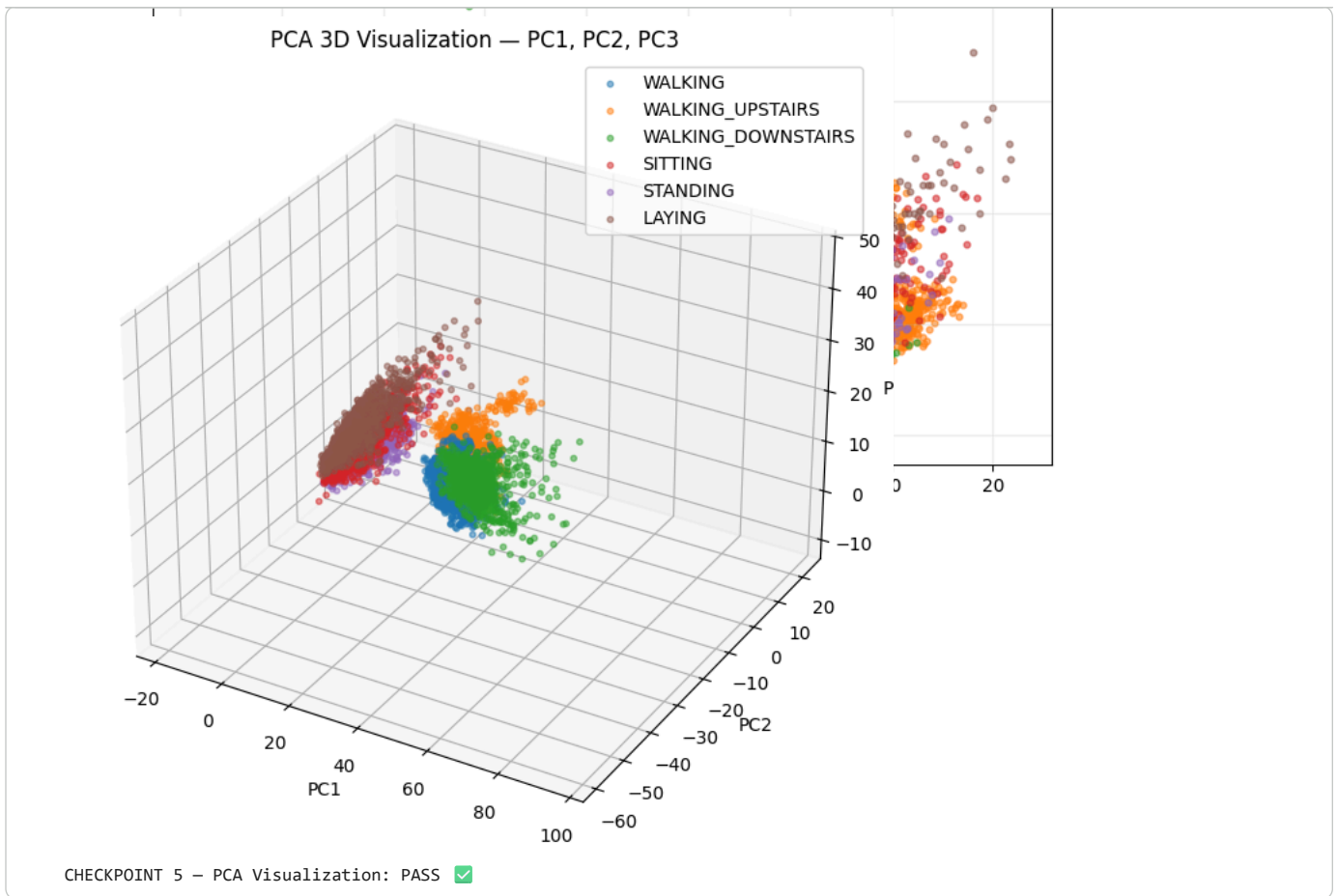
for label in np.unique(y_train):
    mask = y_train == label

    ax.scatter(
        Z_train_3d[mask, 0],
        Z_train_3d[mask, 1],
        Z_train_3d[mask, 2],
        s=10,
        alpha=0.5,
        label=activity_map.get(int(label), str(label))
    )

ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
ax.set_title("PCA 3D Visualization - PC1, PC2, PC3")
ax.legend()
plt.show()
```

```
print("CHECKPOINT 5 - PCA Visualization: PASS ✅")
```





✎ Giai đoạn 7 — Reconstruction Error theo số components

Explained variance cho biết lượng variance giữ lại, nhưng ta cần thêm góc nhìn trực tiếp:

$$[Z_k = XW_k]$$

$$[\hat{X}_k = Z_k W_k^T]$$

$$[MSE(k) = \frac{1}{n} \sum (X - \hat{X}_k)^2]$$

Ta đánh giá **cả train và test**:

- PCA vẫn chỉ fit trên train;
- test dùng cùng scaler và eigenvectors học từ train;
- khi (k) tăng, reconstruction error phải không tăng.

✎ AI PROMPTING LOG — P08: Reconstruction error curve

Prompt:

Tôi đã có `X_train_scaled`, `X_test_scaled` và ma trận eigenvectors `V_all` từ PCA fit trên train. Không được fit lại PCA cho từng k. Hãy đánh giá reconstruction error cho một tập các k đại diện như 1,2,3,5,10,20,30,50,75,100,150,200,300,400,500,561 và tự bổ sung các k đạt 80%,90%,95%,99% variance từ `k_by_threshold`. Với mỗi k, lấy `W_k = V_all[:, :k]`, project train/test bằng `X @ W_k`, reconstruct bằng `Z @ W_k.T`, tính MSE riêng cho train và test. Lưu kết quả vào DataFrame và vẽ reconstruction MSE theo k. Kiểm tra MSE hữu hạn và không tăng khi k tăng; ở k = toàn bộ số feature, train reconstruction MSE phải gần 0. Không dùng sklearn.

Logic cần giữ nguyên: reuse nested PCA basis → reconstruct train/test for multiple k → MSE curve → monotonic validation → full-basis sanity check.

```
# =====
# 17. Reconstruction helpers
# =====
```

```
def project_with_k(X_scaled, V_all, k):
    W_k = V_all[:, :k]
    return X_scaled @ W_k

def reconstruct_with_k(X_scaled, V_all, k):
    W_k = V_all[:, :k]
    Z_k = X_scaled @ W_k
    X_hat = Z_k @ W_k.T
    return X_hat

def reconstruction_mse(X_scaled, X_hat):
    return float(np.mean((X_scaled - X_hat) ** 2))
```

```
# =====
# 18. Reconstruction Error Analysis
# =====
base_k_candidates = [
    1, 2, 3, 5, 10, 20, 30, 50, 75, 100,
    150, 200, 300, 400, 500, n_original_features
]

threshold_k_candidates = list(k_by_threshold.values())

k_candidates = sorted(set(
    k for k in base_k_candidates + threshold_k_candidates
    if 1 <= k <= n_original_features
))

reconstruction_rows = []

for k in k_candidates:
    X_train_hat = reconstruct_with_k(
        X_train_scaled,
        V_all,
        k
    )

    X_test_hat = reconstruct_with_k(
        X_test_scaled,
        V_all,
        k
    )

    train_mse = reconstruction_mse(
        X_train_scaled,
        X_train_hat
    )

    test_mse = reconstruction_mse(
        X_test_scaled,
        X_test_hat
    )

    reconstruction_rows.append({
        "k": k,
        "Train MSE": train_mse,
        "Test MSE": test_mse,
        "Train retained variance": cumulative_variance_all[k - 1],
        "Reduction ratio": 1.0 - (k / n_original_features),
    })

reconstruction_df = pd.DataFrame(reconstruction_rows)

display(reconstruction_df)
```

	k	Train MSE	Test MSE	Train retained variance	Reduction ratio
0	1	4.921883e-01	4.363483e-01	0.507812	0.998217
1	2	4.263815e-01	3.916858e-01	0.573619	0.996435
2	3	3.983171e-01	3.735786e-01	0.601683	0.994652
3	5	3.543947e-01	3.387090e-01	0.645605	0.991087
4	10	2.918443e-01	2.880430e-01	0.708156	0.982175
5	20	2.262535e-01	2.266891e-01	0.773746	0.964349
6	26	1.981950e-01	2.011244e-01	0.801805	0.953654
7	30	1.822040e-01	1.866055e-01	0.817796	0.946524
8	50	1.248164e-01	1.324480e-01	0.875184	0.910873
9	63	9.946549e-02	1.082636e-01	0.900535	0.887701
10	75	8.045218e-02	8.920970e-02	0.919548	0.866310
11	100	5.108496e-02	5.783615e-02	0.948915	0.821747
12	102	4.924418e-02	5.583215e-02	0.950756	0.818182
13	150	1.894627e-02	2.306077e-02	0.981054	0.732620
14	179	9.807321e-03	1.242173e-02	0.990193	0.680927
15	200	6.363887e-03	8.047198e-03	0.993636	0.643494
16	300	9.012385e-04	1.215097e-03	0.999099	0.465241
17	400	2.653673e-05	4.680661e-05	0.999973	0.286988
18	500	2.829675e-17	3.276347e-17	1.000000	0.108734
19	561	2.641135e-30	2.358726e-30	1.000000	0.000000

Next steps: [Generate code with reconstruction_df](#) [New interactive sheet](#)

```
# =====
# 19. Reconstruction Error Plot
# =====
plt.figure(figsize=(11, 6))
plt.plot(
    reconstruction_df["k"],
    reconstruction_df["Train MSE"],
    marker="o",
    label="Train reconstruction MSE"
)
plt.plot(
    reconstruction_df["k"],
    reconstruction_df["Test MSE"],
    marker="o",
    label="Test reconstruction MSE"
)

plt.xlabel("Number of Principal Components (k)")
plt.ylabel("Reconstruction MSE")
plt.title("Reconstruction Error vs Number of Principal Components")
plt.legend()
plt.grid(alpha=0.25)
plt.show()

train_mse_values = reconstruction_df["Train MSE"].to_numpy()
test_mse_values = reconstruction_df["Test MSE"].to_numpy()

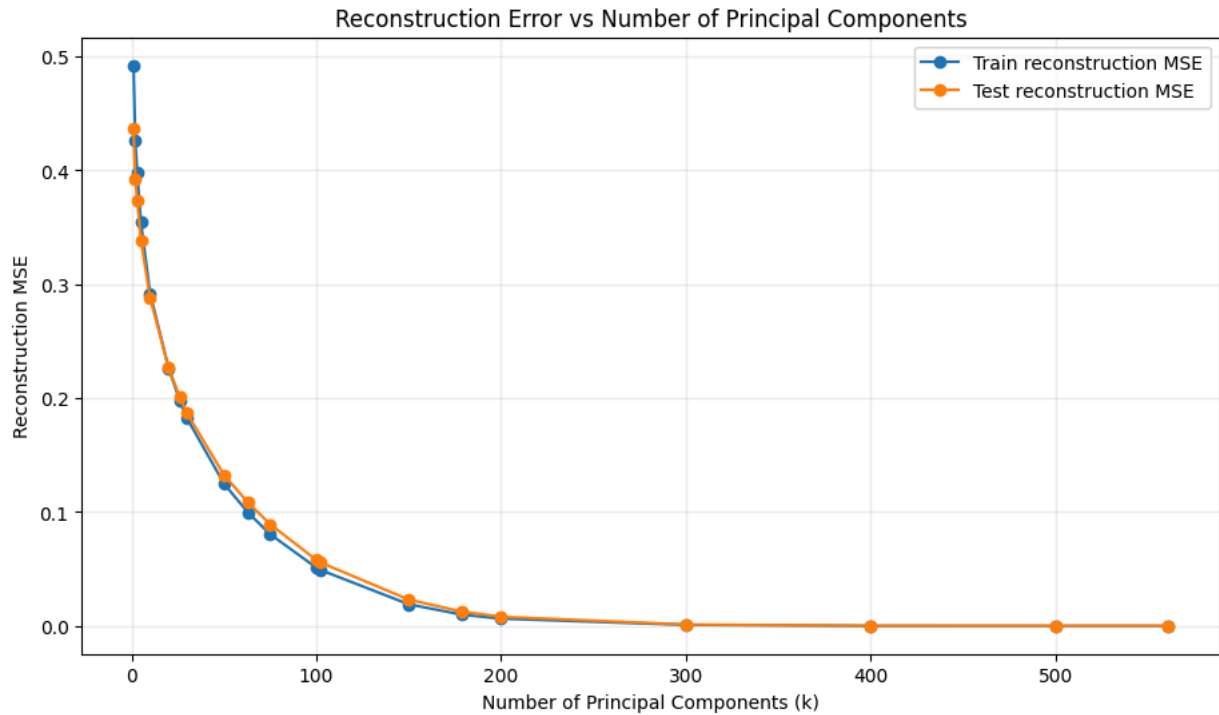
assert np.all(np.isfinite(train_mse_values))
assert np.all(np.isfinite(test_mse_values))
assert np.all(np.diff(train_mse_values) <= 1e-10)
assert np.all(np.diff(test_mse_values) <= 1e-10)

full_k_row = reconstruction_df[
    reconstruction_df["k"] == n_original_features
].iloc[0]

print("Train MSE with all PCs:", full_k_row["Train MSE"])
print("Test MSE with all PCs :", full_k_row["Test MSE"])
```

```
assert full_k_row["Train MSE"] < 1e-20
```

```
print("\nCHECKPOINT 6 – Reconstruction Analysis: PASS ✅")
```



Train MSE with all PCs: 2.6411349198388693e-30

Test MSE with all PCs : 2.3587262766451556e-30

CHECKPOINT 6 – Reconstruction Analysis: PASS ✅

✎ Giai đoạn 8 — Loading Analysis và diễn giải Principal Components

Eigenvector chứa **loading** của các feature gốc trên từng principal component.

Với PC (j), feature có (|loading|) lớn hơn sẽ có đóng góp mạnh hơn vào hướng đó.

Ở đây ta:

1. lấy top features theo (|loading|);
2. giữ cả dấu loading để biết hướng đóng góp;
3. phân tích PC1–PC5;
4. không dùng label.

✎ AI PROMPTING LOG — P09: Feature loading analysis

Prompt:

Tôi có `V_all` là ma trận eigenvectors PCA from scratch với shape `(n_features, n_features)`, trong đó mỗi cột là một principal component, và `feature_names` chứa tên 561 features UCI HAR. Hãy viết hàm trả về top-N feature có absolute loading lớn nhất cho một PC bất kỳ, bao gồm feature index, feature name, signed loading và absolute loading. Dùng hàm để phân tích PC1 đến PC5, mỗi PC lấy top 10 features. Hiển thị bằng rõ ràng và tạo bar chart riêng cho từng PC. Kiểm tra tổng bình phương loading của mỗi eigenvector gần 1 và eigenvectors đã normalized. Không dùng sklearn và không dùng label.

Logic cần giữ nguyên: eigenvector column → absolute loading rank → preserve sign/name → top-feature interpretation → normalization sanity check.

```
# =====
# 20. Loading Analysis
# =====
def top_loadings_for_pc(
    V_all,
```

```

feature_names,
pc_index,
top_n=10
):
    # pc_index dùng zero-based indexing: 0 -> PC1, 1 -> PC2, ...
    loadings = V_all[:, pc_index]

    top_indices = np.argsort(
        np.abs(loadings)
    )[::-1][:top_n]

    rows = []

    for idx in top_indices:
        rows.append({
            "PC": f"PC{pc_index + 1}",
            "Feature index": int(idx),
            "Feature name": feature_names[idx],
            "Loading": float(loadings[idx]),
            "|Loading|": float(abs(loadings[idx])),
        })

    return pd.DataFrame(rows)

loading_tables = {}

for pc_idx in range(5):
    df_pc = top_loadings_for_pc(
        V_all,
        feature_names,
        pc_idx,
        top_n=10
    )

    loading_tables[f"PC{pc_idx + 1}"] = df_pc

    print(f"\n===== TOP LOADINGS - PC{pc_idx + 1} =====")
    display(df_pc)

    norm_sq = float(np.sum(V_all[:, pc_idx] ** 2))
    print(f"Sum of squared loadings PC{pc_idx + 1}: {norm_sq}")

    assert abs(norm_sq - 1.0) < 1e-10

print("\nLoading normalization checks: PASS ✅")

```


Next steps:

Generate code with df_pc

New interactive sheet

Generate code with df_pc

New interactive sheet

Generate code with df_pc

===== TOP LOADINGS – PC1 =====

	PC	Feature index	Feature name	Loading	Loading	
0	PC1	280	fBodyAcc-sma()	0.058571	0.058571	
1	PC1	359	fBodyAccJerk-sma()	0.058568	0.058568	
2	PC1	95	tBodyAccJerk-sma()	0.058538	0.058538	
3	PC1	438	fBodyGyro-sma()	0.058531	0.058531	
4	PC1	231	tBodyAccJerkMag-sma()	0.058493	0.058493	
5	PC1	226	tBodyAccJerkMag-mean()	0.058493	0.058493	
6	PC1	515	fBodyBodyAccJerkMag-mean()	0.058128	0.058128	
7	PC1	520	fBodyBodyAccJerkMag-sma()	0.058128	0.058128	
8	PC1	228	tBodyAccJerkMag-mad()	0.058017	0.058017	
9	PC1	227	tBodyAccJerkMag-std()	0.057985	0.057985	

Sum of squared loadings PC1: 1.0000000000000002

===== TOP LOADINGS – PC2 =====

	PC	Feature index	Feature name	Loading	Loading	
0	PC2	295	fBodyAcc-meanFreq()-Z	-0.122385	0.122385	
1	PC2	248	tBodyGyroMag-arCoeff()1	-0.119783	0.119783	
2	PC2	209	tBodyAccMag-arCoeff()1	-0.118162	0.118162	

```
# =====
# 21. Loading bar charts – PC1 đến PC5
# =====
for pc_name, df_pc in loading_tables.items():
    plot_df = df_pc.iloc[:, :-1]

    plt.figure(figsize=(10, 6))
    plt.barh(
        plot_df["Feature name"],
        plot_df["Loading"]
    )
    plt.xlabel("Signed Loading")
    plt.ylabel("Feature")
    plt.title(f"Top 10 Feature Loadings – {pc_name}")
    plt.grid(axis="x", alpha=0.2)
    plt.show()

print("CHECKPOINT 7 – Loading Analysis: PASS ✅")

2 PC3          63  tGravityAcc-entropy()-Y  0.109555  0.109555
3 PC3          58  tGravityAcc-energy()-Z    0.109078  0.109078
4 PC3          52    tGravityAcc-min()-X   -0.108219  0.108219
5 PC3         558    angle(X,gravityMean)  0.107508  0.107508
6 PC3          43    tGravityAcc-std()-X    0.104384  0.104384
7 PC3          40    tGravityAcc-mean()-X   -0.104127  0.104127
8 PC3          46    tGravityAcc-mad()-X    0.104113  0.104113
9 PC3          59    tGravityAcc-iqr()-X    0.102008  0.102008

Sum of squared loadings PC3: 0.9999999999999998

===== TOP LOADINGS – PC4 =====

0 PC4          51  tGravityAcc-max()-Z   -0.143971  0.143971
1 PC4          42  tGravityAcc-mean()-Z   -0.141018  0.141018
2 PC4         560    angle(Z,gravityMean)  0.139935  0.139935
3 PC4          54    tGravityAcc-min()-Z   -0.136219  0.136219
4 PC4          52    tGravityAcc-min()-X    0.127389  0.127389
5 PC4          40    tGravityAcc-mean()-X    0.124531  0.124531
6 PC4          306  fBodyAcc-kurtosis()-X  0.123734  0.123734
```

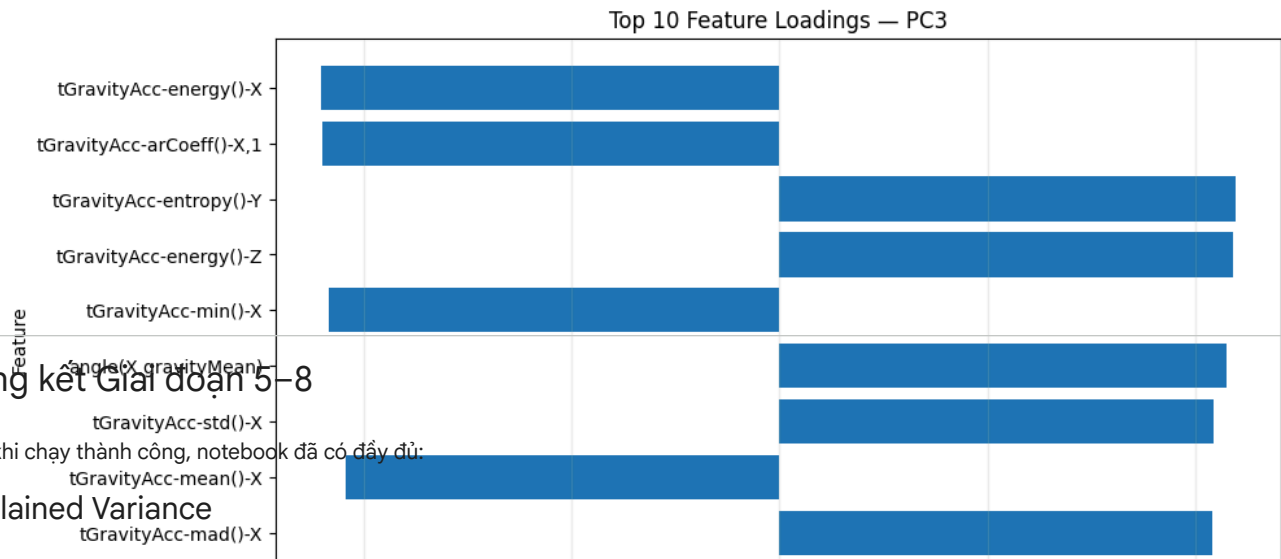
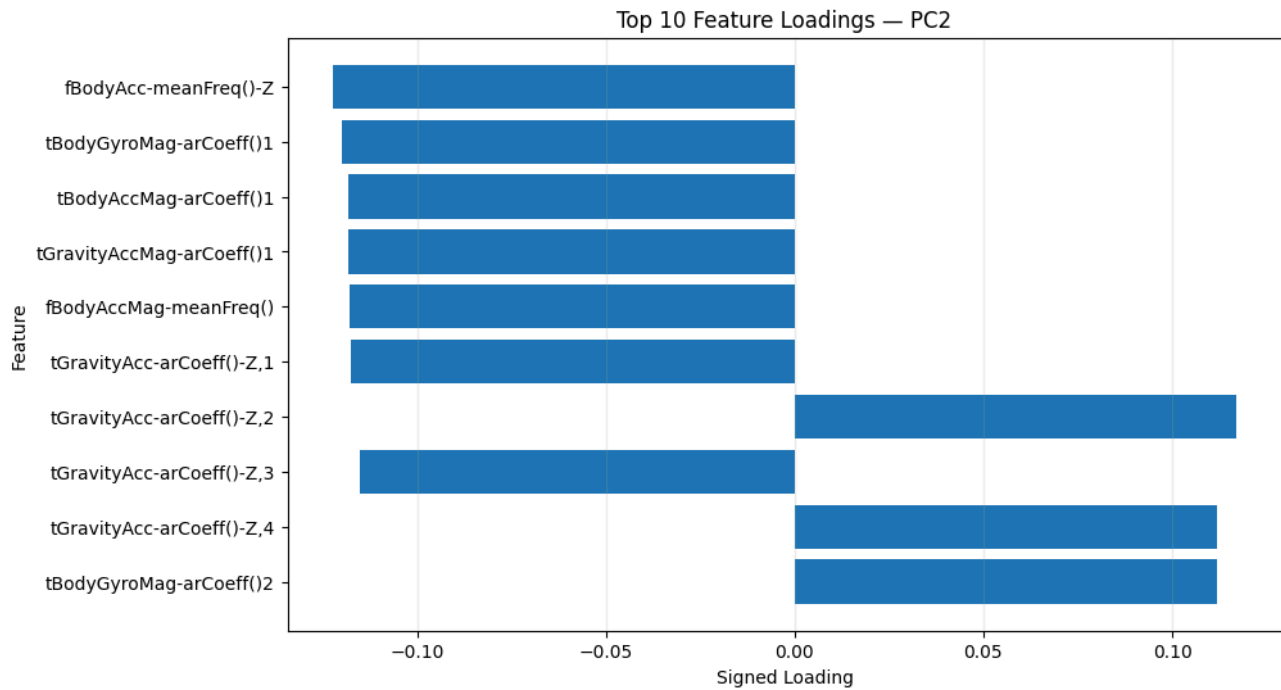
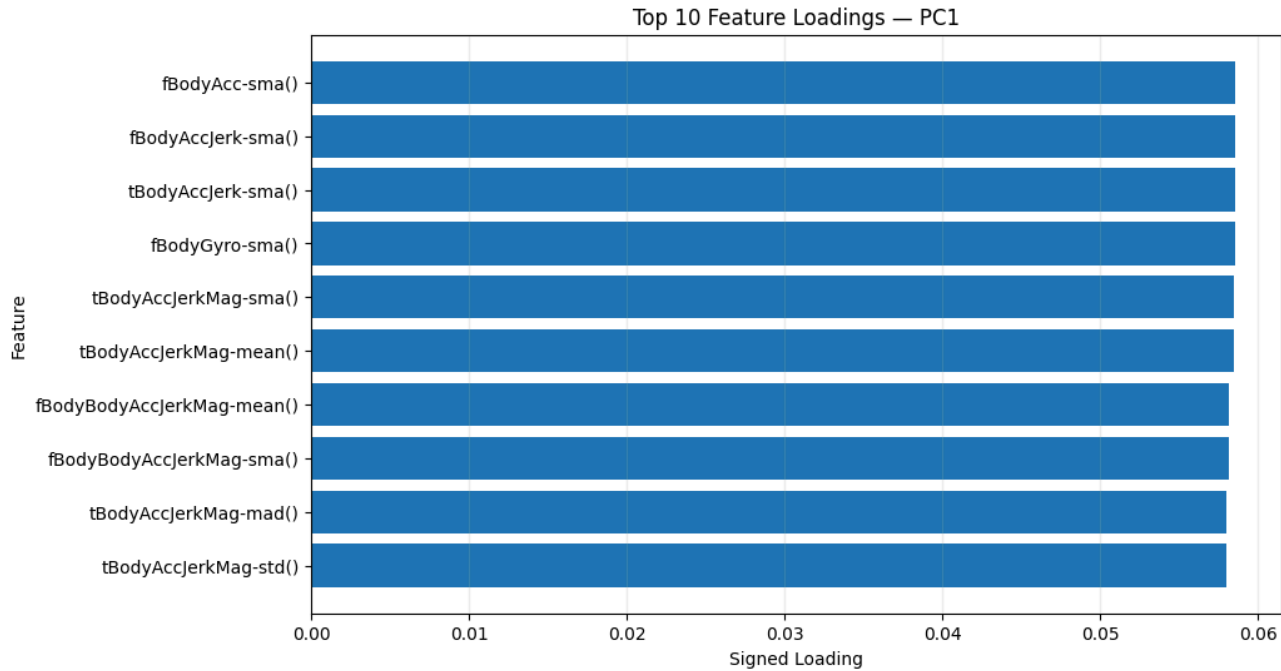
```
6 PC4          290  fBodyAcc-skewness()-X -0.120724  0.120724
7 PC4          297  fBodyAcc-kurtosis()-X -0.122033  0.122033
8 PC4           56  tGravityAcc-energy()-X  0.121121  0.121121
9 PC4           49  tGravityAcc-max()-X  0.120072  0.120072
Sum of squared loadings PC4: 0.9999999999999986
```

===== TOP LOADINGS - PC5 =====

	PC	Feature index	Feature name	Loading	Loading	
0	PC5	558	angle(X.gravityMean)	0.167569	0.167569	
1	PC5	56	tGravityAcc-energy()-X	-0.165242	0.165242	
2	PC5	49	tGravityAcc-max()-X	-0.162362	0.162362	
3	PC5	40	tGravityAcc-mean()-X	-0.160478	0.160478	
4	PC5	52	tGravityAcc-min()-X	-0.158584	0.158584	
5	PC5	57	tGravityAcc-energy()-Y	0.148915	0.148915	
6	PC5	71	tGravityAcc-arCoeff()-Y,3	0.140883	0.140883	
7	PC5	70	tGravityAcc-arCoeff()-Y,2	-0.140379	0.140379	
8	PC5	72	tGravityAcc-arCoeff()-Y,4	-0.138324	0.138324	
9	PC5	69	tGravityAcc-arCoeff()-Y,1	0.137638	0.137638	

Sum of squared loadings PC5: 0.9999999999999991

Loading normalization checks: PASS 



Tổng kết Giai đoạn 5-8

Sau khi chạy thành công, notebook đã có đầy đủ:

Explained Variance

- Scree Plot;
- cumulative explained variance;
- (k80, k90, k95, k99);
- tỷ lệ giảm số chiều.



PCA Visualization

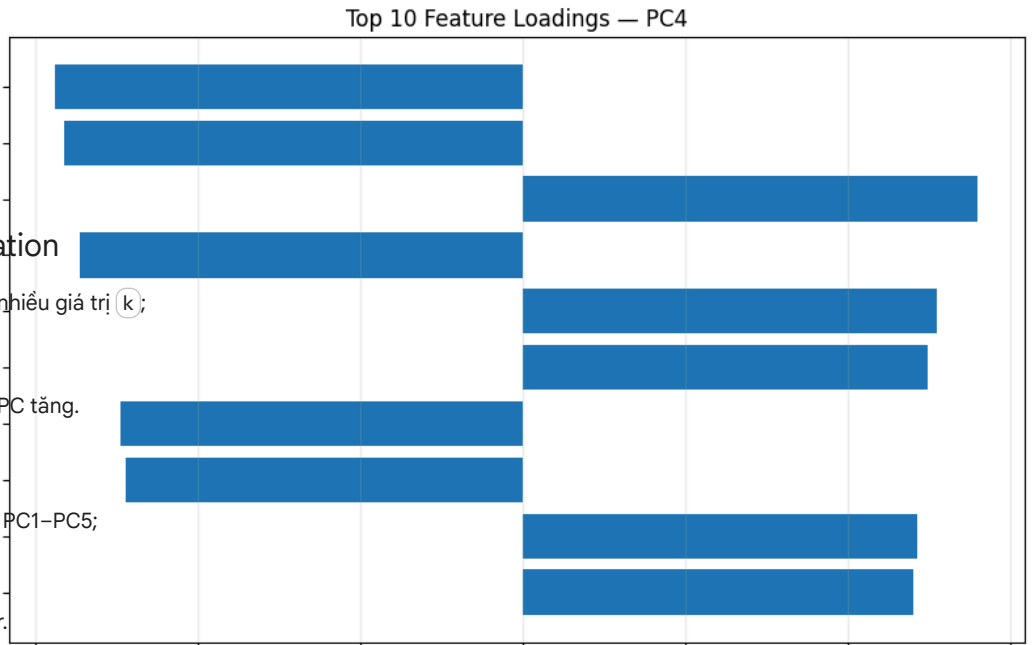
- PC1 vs PC2;
- PC1 vs PC3;
- PC2 vs PC3;
- PC1-PC2-PC3 3D.

Reconstruction Evaluation

- reconstruction MSE theo nhiều giá trị (k);
- train MSE;
- test MSE;
- kiểm tra MSE giảm khi số PC tăng.

Loading Analysis

- top feature đóng góp vào PC1-PC5;
- signed loading;
- absolute loading;
- kiểm tra norm eigenvector.



Sau khi các checkpoint 4-7 đều PASS, có thể chuyển sang **Giai đoạn 9: lựa chọn số Principal Components cuối cùng**, rồi xuất `X_train_pca90`, `X_test_pca90`, `X_train_pca95`, `X_test_pca95` để chuẩn bị cho classification.

Top 10 Feature Loadings — PC5

✓ Giai đoạn 9 — Lựa chọn cấu hình PCA cuối cùng

Các checkpoint trước đã xác nhận PCA hoạt động đúng về mặt toán học. Bây giờ ta quyết định **cấu hình nào sẽ được mang sang classification**.

Không nên gọi một giá trị (k) là "tối ưu tuyệt đối" chỉ dựa trên PCA, vì hiệu quả cuối cùng còn phải được xác nhận bằng downstream classifier.

Ta giữ hai cấu hình ứng viên:

- **PCA90**: ưu tiên giảm chiều mạnh hơn.
- **PCA95**: ưu tiên bảo toàn thông tin nhiều hơn.

`PCA95` được dùng làm cấu hình **primary** trước classification; `PCA90` là **compression-oriented alternative**.

✓ AI PROMPTING LOG — P10: Final PCA candidate selection

Prompt: tGravityAcc-arCoeff()-Y,1

Tôi đã chạy PCA from scratch trên UCI HAR và có `k_by_threshold`, `cumulative_variance_all`, `reconstruction_df`, `n_original_features`. Hãy tạo bảng so sánh PCA80, PCA90, PCA95, PCA99 với: target variance, k, actual retained variance, dimensions reduced, reduction ratio, train reconstruction MSE và test reconstruction MSE. Không dùng sklearn. Không tuyên bố một k là tối ưu tuyệt đối trước classification. Đặt PCA95 là cấu hình primary vì ưu tiên bảo toàn thông tin, PCA90 là alternative vì ưu tiên compression. Kiểm tra PCA95 có retained variance ≥ 0.95 , PCA90 ≥ 0.90 và $k_{90} \leq k_{95}$. In giải thích ngắn về trade-off.

Logic cần giữ nguyên: threshold candidates $\rightarrow$ reconstruction metrics $\rightarrow$ trade-off minh bạch $\rightarrow$ primary/alternative, không overclaim.

```
# =====
# 22. Candidate comparison table
# =====
candidate_rows = []

for threshold in [0.80, 0.90, 0.95, 0.99]:
    k = int(k_by_threshold[threshold])
```

```

rec_match = reconstruction_df[
    reconstruction_df["k"] == k
]

if rec_match.empty:
    X_train_hat = reconstruct_with_k(X_train_scaled, V_all, k)
    X_test_hat = reconstruct_with_k(X_test_scaled, V_all, k)

    train_mse = reconstruction_mse(X_train_scaled, X_train_hat)
    test_mse = reconstruction_mse(X_test_scaled, X_test_hat)
else:
    train_mse = float(rec_match.iloc[0]["Train MSE"])
    test_mse = float(rec_match.iloc[0]["Test MSE"])

retained = float(cumulative_variance_all[k - 1])
reduction_ratio = 1.0 - (k / n_original_features)

candidate_rows.append({
    "Configuration": f"PCA{int(threshold * 100)}",
    "Target variance": threshold,
    "k": k,
    "Actual retained variance": retained,
    "Dimensions reduced": n_original_features - k,
    "Reduction ratio": reduction_ratio,
    "Train reconstruction MSE": train_mse,
    "Test reconstruction MSE": test_mse,
})

pca_candidate_df = pd.DataFrame(candidate_rows)
display(pca_candidate_df)

k90 = int(k_by_threshold[0.90])
k95 = int(k_by_threshold[0.95])

assert k90 <= k95
assert cumulative_variance_all[k90 - 1] >= 0.90
assert cumulative_variance_all[k95 - 1] >= 0.95

PRIMARY_CONFIG = "PCA95"
ALTERNATIVE_CONFIG = "PCA90"

print("Primary configuration      :", PRIMARY_CONFIG, f"(k={k95})")
print("Alternative configuration:", ALTERNATIVE_CONFIG, f"(k={k90})")
print()
print("Interpretation:")
print("- PCA95 ưu tiên bảo toàn thông tin trước classification.")
print("- PCA90 ưu tiên giảm chiều mạnh hơn.")
print("- Chưa gọi cấu hình nào là tối ưu tuyệt đối trước downstream evaluation.")

print("\nCHECKPOINT 8 – Candidate Selection: PASS ✅")

```

	Configuration	Target variance	k	Actual retained variance	Dimensions reduced	Reduction ratio	Train reconstruction MSE	Test reconstruction MSE
0	PCA80	0.80	26	0.801805	535	0.953654	0.198195	0.201124
1	PCA90	0.90	63	0.900535	498	0.887701	0.099465	0.108264
2	PCA95	0.95	102	0.950756	459	0.818182	0.049244	0.055832
3	PCA99	0.99	179	0.990193	382	0.680927	0.009807	0.012422

Primary configuration : PCA95 (k=102)
 Alternative configuration: PCA90 (k=63)

Interpretation:
 - PCA95 ưu tiên bảo toàn thông tin trước classification.
 - PCA90 ưu tiên giảm chiều mạnh hơn.
 - Chưa gọi cấu hình nào là tối ưu tuyệt đối trước downstream evaluation.

Next steps: [Generate code with pca_candidate_df](#)

[New interactive sheet](#)

✧ Giai đoạn 10 — Tạo dữ liệu PCA90/PCA95 và kiểm tra cuối